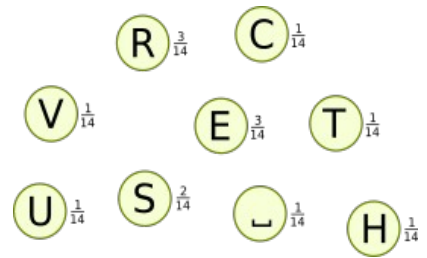


## Das Huffman-Kompressionsverfahren

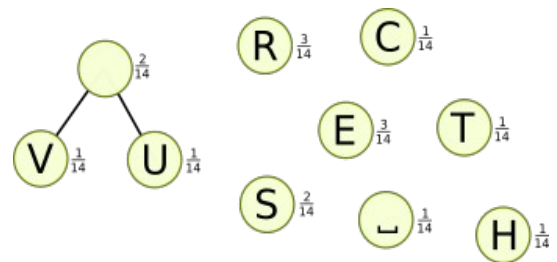
Der Huffman-Algorithmus (nach David A. Huffman, der ihn 1952 publiziert hat) dient zur Kompression beliebiger Daten, welche aus einem Alphabet von Symbolen verfügen, die unterschiedlich oft auftreten können. Ein gutes Beispiel für solche Daten ist ein gewöhnlicher Text in einer beliebigen Sprache. Der Huffman-Algorithmus findet den optimalen binären Code für jedes Symbol, wobei «optimal» bedeutet, dass der resultierende Datenstrom möglichst wenig Redundanz enthält (und damit möglichst wenige Bits enthält).

### Anleitung

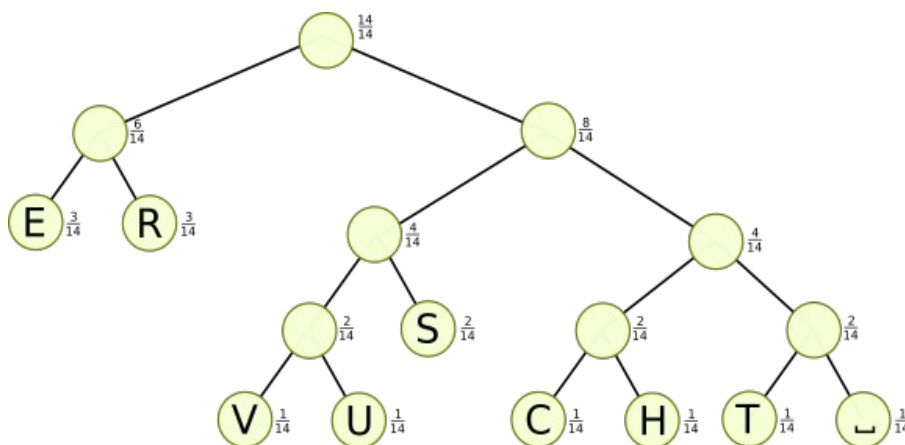
1. Bestimme die relative Häufigkeit jedes Symbols in deinem Datenstrom. Für einen Text bedeutet dies, dass du einfach die Häufigkeiten der im Text vorkommenden Zeichen auszählst und jede Häufigkeit durch die Gesamtzahl der Zeichen dividierst. Für den (sehr kurzen) Text «ERSTER VERSUCH» ergeben sich die rechts abgebildeten relativen Häufigkeiten. Beachte, dass auch das Leerzeichen ein Zeichen ist! Rechts haben wir jeden Buchstaben in einen Kreis gepackt; wir nennen diese Kreise «Knoten» und Verbindungen zwischen ihnen «Kanten».



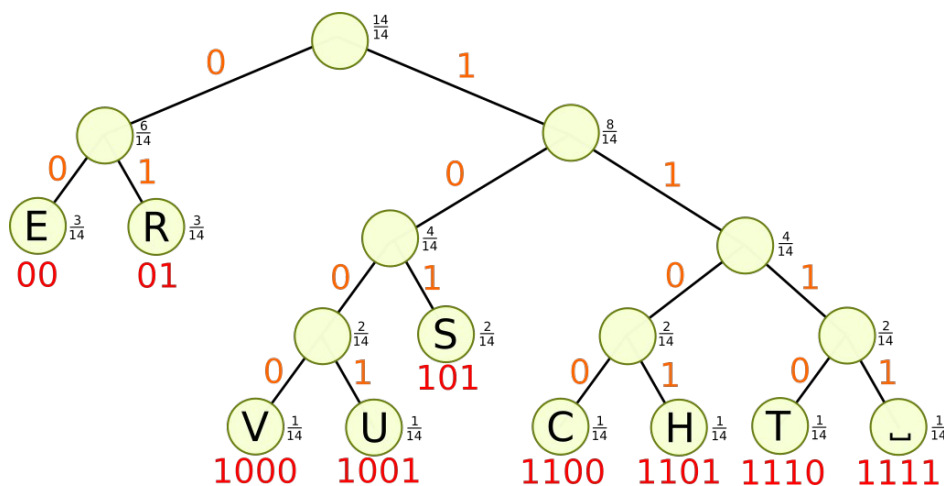
2. Wähle die beiden Bäume, deren Wurzelknoten mit den zwei kleinsten Zahlen beschriftet sind, und setze sie zu einem Baum zusammen, dessen neu entstandenen Wurzelknoten du mit der Summe der Zahlen der beiden Teilbäume anschreibst. Wenn es mehrere Möglichkeiten gibt, wähle die Bäume, die am wenigsten hoch sind (Fachsprache: diejenigen mit der geringsten Tiefe). Hinweis: Auch ein einzelner Knoten ist ein Baum; er besteht halt einfach nur aus dem Wurzelknoten (Tiefe 0). Wähle also z.B. die Knoten «V» und «U»; dann erhältst du die Situation rechts.



3. Wiederhole Schritt 2 so lange, bis alle Teilbäume zu einem einzigen Baum geworden sind. Auf dem *Zusatzblatt* siehst du diesen Prozess Schritt für Schritt; hier ist das Ergebnis:



4. Schreibe die Kanten (also die Verbindungen von den Eltern- zu den Kindknoten) mit 0 und 1 an; die Kante zum linken Kind immer mit 0, die Kante zum rechten Kind mit 1. Folge dann für jedes Blatt im Baum (also jeden Buchstaben) dem Pfad von der Wurzel bis zum Buchstaben und merke dir die Folge von 0 und 1, die du passierst. Den Buchstaben kannst du anschliessend mit dieser Bitfolge anschreiben. Auf der nächsten Seite ist das gemacht.



5. Nun hast du für jedes Symbol einen Binärcode. Der Huffman-Algorithmus stellt sicher, dass die häufigeren Zeichen kürzere Codes erhalten. Um also «ERSTER VERSUCH» zu codieren, benötigst du nicht mehr wie bei der ASCII-Codierung 7 oder 8 Bits pro Zeichen, sondern je nachdem, wie häufig das Zeichen ist, mehr oder weniger Bits. Für «ERSTER VERSUCH» ergibt sich die Bitfolge 00 01 101 1110 00 01 1111 1000 00 01 101 1001 1100 1101. Die Lücken sind nur zur besseren Lesbarkeit da, tatsächlich gibt es natürlich keine Lücken, sondern nur eine Folge bzw einen Strom von Binärziffern:

000110111100001111110000001101100111001101

Das sind insgesamt 42 Bits; verglichen mit Codierungen wie ASCII oder UTF-8, die 8 Bits pro Zeichen benötigen, d.h. 112 Bits für den String «ERSTER VERSUCH», haben wir einen Kompressionsfaktor von 0.35 bzw eine Kompressionsrate von 2.8 erreicht.

6. Um den Bitstrom 000110111100001111110000001101100111001101 wieder zu decodieren, benötigst du den obigen Baum. Du beginnst an der Wurzel und folgst für jedes Bit im Strom einfach den entsprechend beschrifteten Kanten, bis du einen Buchstabenknoten erreichst; dann springst du wieder zurück zur Wurzel.

## Aufgaben

1. Ist das Huffman-Verfahren eine verlustfreie oder eine verlustbehaftete Kompressionsmethode?
2. Schau dir, wenn du es noch nicht gemacht hast, den Prozess zur Herstellung des Baums auf dem Zusatzblatt an, bis du den Prozess verstehst.
3. Erstelle den Baum für den Text «HUFFMAN WAR EIN SCHLAUER KERL» und gib die Bitfolge für den so codierten Text an.
4. Berechne den in Aufgabe 3 erreichten Kompressionsfaktor im Vergleich zu einer Codierung, die ein Byte pro Zeichen verwendet.
5. Bonusaufgabe (schwierig): In der Realität werden die Codierungsbäume natürlich nicht von Hand von Menschen erstellt, sondern automatisch von Kompressionsprogrammen. Weil der Codierungsbaum für jeden Text anders aussieht, muss alles nötige, um ihn zu erzeugen, der codierten Nachricht beigelegt werden, damit sie wieder decodiert werden kann. Wie könntest du die nötigen Informationen zur Herstellung des Baums möglichst effizient angeben? Welche Informationen benötigt man überhaupt, um den Baum zu konstruieren?